Timeline: Adding the time dimension to spreadsheets

ANONYMOUS AUTHOR(S)

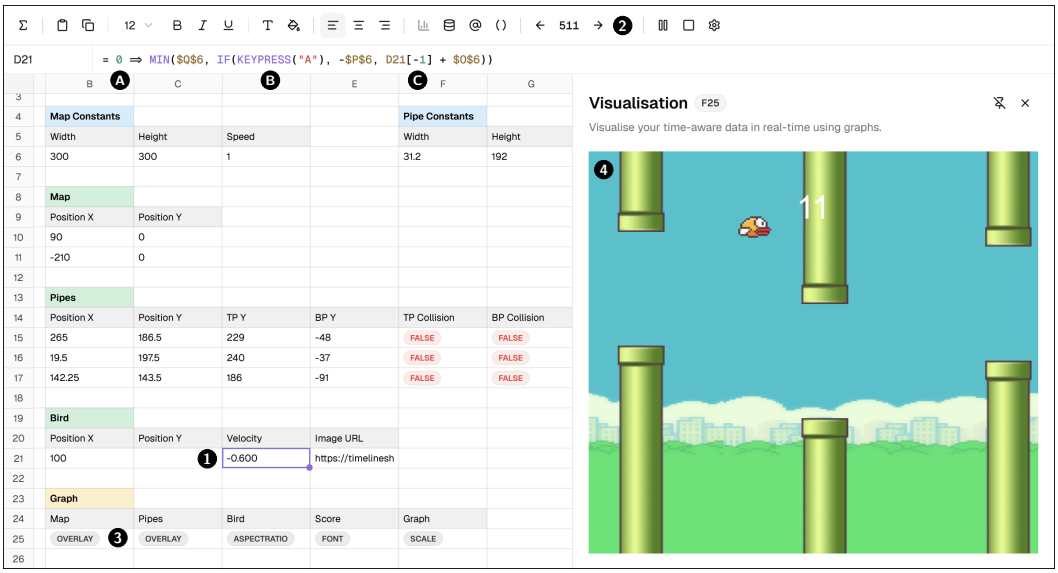

Fig. 1. A flappy bird game clone in Timeline. ❶ The selected formula computes the Y velocity in the current step. ❷ The system uses discrete time. ❸ Visualizations are built using a composable grammar of graphics and ❹ can be displayed inline or in a separate panel. The formula for the velocity Ⓐ specifies an initial value, Ⓑ reacts to user-interface events, and Ⓒ accesses a previous value to compute the current one.

Spreadsheets make it easy to express computations over two-dimensional data, but two dimensions are not enough to express rich computations with time such as physics simulations, agent-based models, analyses of financial data, or interactive systems. We present Timeline, a system that adds discrete time to spreadsheets.

The remarkable insight from our work is that a large number of advanced programming language concepts can be directly applied in the context of spreadsheets with time. Timeline draws from *dataflow languages* to express computations over time, *coeffect systems* to ensure bounded memory usage, *functional reactive programming* to support interactivity, *grammars of graphics* to support composable visualizations, and *typed holes* for inserting cell references in the formula editor.

In this paper, we provide an overview of the Timeline design and discuss how it adapts the aforementioned programming language innovations for the context of spreadsheets. We formalize the evaluation of spreadsheets with discrete time through a *core calculus*, describe a coeffect-based static analysis that determines the required number of past values and prove that the optimization is sound. More broadly, this paper shows that established programming language ideas can often be productively used outside of their original domain.

CCS Concepts: • **Software and its engineering → Very high level languages**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-735X/2026/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Spreadsheets are sometimes referred to as "the most widely used programming systems in the world" [2]. Their uses range from personal productivity tools and decision making support systems to scientific analytical instruments and financial reporting systems [27]. However, spreadsheets are first-order, limited to tabular data structures, and lack abstraction mechanisms.

Many extensions to the core spreadsheet model have been proposed, generalising the tabular structure to non-tabular forms [8, 15], adding support for objects [41], or incorporating support for streaming data to use spreadsheets as the back-end for web development [19, 20]. Abstraction mechanism developed for spreadsheets include sheet-defined functions [34, 51] and gridlets [33], while programming by example [28] simplifies data wrangling within spreadsheets.

The above extensions show that extending spreadsheets is a difficult balancing act. How can we make spreadsheets more expressive, while preserving their liveness, directness and usability [30]? The aim of Timeline is to extend spreadsheets with a built-in notion of discrete time, making it possible to use spreadsheets in a range of domains that are difficult to tackle with only two dimensions [37] such as physics simulations, agent-based modelling [1], financial data analysis, or simple interactive applications. As illustrated in Figure 1, Timeline adds an implicit discrete time to the spreadsheet programming model and extends the formula language to allow references to previous values and user interface events, but otherwise preserves the standard spreadsheet model.

Adapting programming langauge research. Although Timeline is interesting as an exploration of the design, expressive power and usability of spreadsheets with discrete time, this paper considers it primarily from the technical perspective of programming language design and implementation. The remarkable result is that a large number of concepts and techniques developed in the context of programming language research are directly applicable to spreadsheets.

The key addition to Timeline is a built-in notion of discrete time. Formulas in cells can refer to other cells in the current time step (e.g. `G19`), but also values computed in earlier steps (e.g. `F19[-1]`). The language design and runtime implementation of Timeline draws inspiration from research on *synchronous dataflow languages* [9, 29]. To ensure that simulations and interactive applications execute using bounded memory whenever possible, we augment the runtime with a program analysis based on *coeffects* [47, 48]. The support for discrete times also makes it possible to use the spreadsheet paradigm for building interactive applications. Here, we draw on the *functional reactive programming* paradigm [25, 53]. Although not strictly related to time, Timeline also adopts a *composable data visualization* model [38, 46, 56]. Finally, Timeline also uses a notion of *typed holes* [43] to determine when the user may want to insert a cell reference during formula editing.

Contributions. This paper presents the design and implementation of a spreadsheet system with discrete time and captures its essence through a core calculus.

- We provide an overview of the Timeline design through a simple physics simulation example (§ 2) and examine the expressive power of the programming model through a number of case studies covering agent-based modelling, physics simulations and finance (§ 5).
- We discuss how how the design and implementation of Timeline adapts advanced programming language concepts for the context of spreadsheets (§ 3), including synchronous languages, coeffects, functional reactive programming, visualization grammars and typed holes.
- We present a core calculus of the Timeline execution model (§ 4), which captures the way previous values are stored during the spreadsheet evaluation. We define a mechanism for checking references to earlier time steps (§4.4), formalize a space optimization based on coeffects (§4.5) and prove their soundness (§4.6).

2 Overview

Although the focus of this paper is on the programming language aspects of Timeline, we first briefly review the design motivation for adding time to spreadsheets. We then introduce Timeline through an example that leverages the time dimension and illustrates the capabilities of the system.

Design Motivation. To motivate the design of Timeline, we conducted formative interviews with spreadsheet experts, including some with major applied experience [24], and we analysed models created using modelling languages with discrete time [52]. The analysis shows that tasks involving discrete time often require three dimensions. In agent-based modeling [1], a simulation consists of multiple agents with multiple attributes that evolve over time. Physics simulations that require numerical solution [52] also require tracking multiple attributes of multiple entities over time. Financial analyses of historical data have the same structure.

Although conceptually simple, such modelling problems are difficult to represent in the traditional spreadsheet paradigm. Using one dimension for time forces users to seek workarounds for encoding the remaining two dimensions, for example by splitting attributes of individual agents across separate tables. In practice, analysts are reluctant to use one of the two spreadsheet dimensions for time. This has been confirmed by a formative interview participant, who noted that *"once you set the rows to time steps, you have lost a dimension of your spreadsheet"* and *"if you use one [dimension] for time, everything else becomes miserable."* Timeline aims to avoid such misery.

Motivating Example. To demonstrate Timeline, we consider a simple numerical physics simulation of cannonball firing. In each step, the simulation computes the new cannonball position using the position in the previous step and the current velocity. The initial velocity is computed from the initial angle and firing speed and then updated using gravity and air drag, which itself depends on the velocity in the previous step. The simulation runs until the cannonballs hit the ground.

There are three dimensions in the example: we have multiple cannonballs, with multiple attributes, updated over multiple time steps. Implementing the simulation in a conventional spreadsheet is possible, but challenging. One option is to have multiple tables, one for each attribute (X and Y positions, X and Y velocities, X and Y air drags) with columns for individual cannonballs and rows for time steps. This results in a complex structure with computation spread across 8 tables, but it allows adding cannonballs and time steps using drag right and drag down operations (the user selects the existing table and drags the selection to copy formulas to further empty cells).

Implementing Cannonball Simulation. In Timeline, we can implement the cannonball simulation using a single table. We use this example to introduce the key features of the system.

The calculation of the projectile motion is done in a single table that records attributes of individual cannonballs over time (Figure 2, ❶). The table includes the current position (`B3`, `C3`), velocity (`F3`, `G3`) and air drag (`H3`, `I3`) as well as the initial angle and speed (`D3`, `E3`). We discuss the calculation for the first cannonball (row 3). Other cannonballs are added by selecting the last row and using the drag down operation to copy its formulas, automatically updating the row references.

To specify the position of a cannonball, we need to give the initial position and then specify how to compute the position in the current time step in terms of the value in the previous time step. This illustrates two extensions that Timeline makes to the core spreadsheet model. First, the => operator specifies an initial value for a cell. Second, the notation `C3[-1]` lets us access the value of the cell `C3` in a previous time step. For cannonball, the initial X and Y positions are 0, the position in later steps is computed by adding the current X and Y velocities to the previous position and the computation stops if the cannonball hits the ground:

```
/* Position X */ B3 = 0 => IF(C3[-1] < 0, B3[-1], B3[-1] + F3)
/* Position Y */ C3 = 0 => IF(C3[-1] < 0, C3[-1], C3[-1] + G3)
```

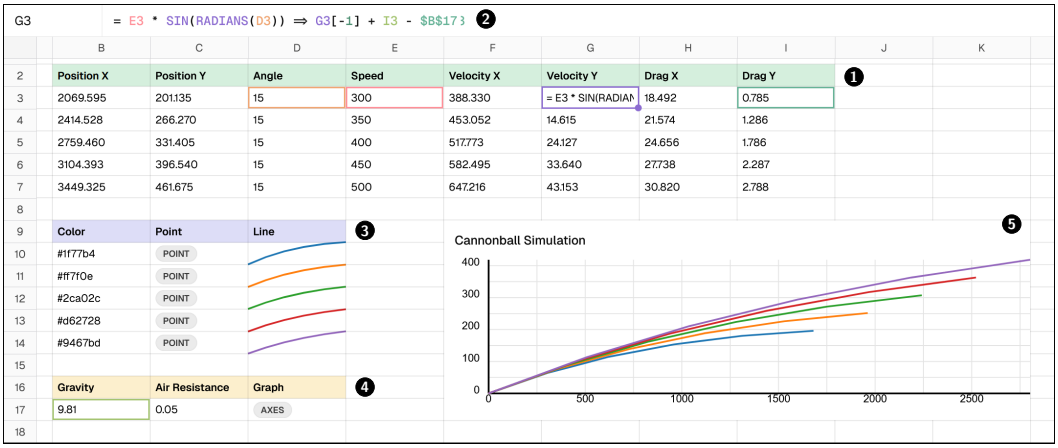

Fig. 2. Numerical cannonball firing simulation in Timeline. ❶ Single table records attributes of multiple cannonballs over time. ❷ The attributes are computed based on current and previous values of other attributes. ❸ A separate table plots past positions as lines. ❹ A table with configuration and composed final chart. ❺ The composed chart is then embedded in the sheet.

The calculation of the cannonball velocity again uses =>. This time, the initial value is a formula using the firing speed and angle (E3, D3). The velocity is then computed from the value in the previous time step and air drag. For the Y velocity, we also subtract gravity, which is a fixed cell reference (B17) that is not updated during the drag down operation. Air drag is obtained by multiplying the previous X and Y velocities by the air resistance (C17):

```
/* Velocity X */ F3 = E3 * COS(RADIANS(D3)) => F3[-1] + H3
/* Velocity Y */ G3 = E3 * SIN(RADIANS(D3)) => G3[-1] + I3 - $B$17
/* Air Drag X */ H3 = F3[-1] * $C$17
/* Air Drag Y */ I3 = G3[-1] * $C$17
```

Note that the calculation of air drag does not require specifying an initial value, because there are no references to previous values of H3 and I3. The only references to the cells are in the velocity computation, but those refer to the current value. The air drag computation accesses velocity at the previous time step, which always exists because velocity defines an initial value using =>. As in standard spreadsheet systems, references between current values of cells need to be acyclic. References to previous time steps do not introduce a cycle.

An interesting problem is how many past values need to be kept by the system. In the computation described so far, we access one previous value of cells B3, C3 and F3, G3 using the [-1] notation. It is thus sufficient to keep one past value for these four cells. For the chart (Figure 2, ❺) we need to store all previous values of the cells that define the line. We discuss this aspect in detail in §3.2.

Composable Data Visualization. Data visualizations in standard spreadsheet systems exist separately of the core spreadsheet model. They are created through graphical user interface where the user specifies data sources and other chart attributes. Timeline takes inspiration from research on spreadsheet-based and functional composable visualizations [38, 46] and allows constructing of visualizations directly in the spreadsheet.

The cannonball firing chart (Figure 2, ❺) is obtained by constructing line charts that represent the movement of individual cannonballs over time and composing the individual lines (❸ and ❹). This is done in a separate table that contains one chart for each cannonball. Table row at index 10 (C10, D10) constructs a line chart for cannonball at row index 3 (B3, C3):

```
/* Point */ C10 = POINT(B3, C3)
/* Line */  D10 = GRAPH(STROKECOLOR(B10, LINE(C10[:-STEP()])))
/* Graph */ D17 = AXES("left bottom", OVERLAY(D10:D14))
```

The example uses a function POINT that constructs a value representing a two-dimensional point and a number of functions for constructing and transforming charts including LINE, OVERLAY and AXES. The latter are inspired by composable data visualizations [46]. LINE takes a range of points and constructs a primitive chart shape representing a line; OVERLAY takes a range of chart shapes and constructs a single chart shape consisting of the individual shapes; AXES attaches axes with labels to a given chart shape. The addition requires adding points and chart shapes as two additional primitive values to the core spreadsheet model. In systems that support richer data structures [15, 22, 42, 58], a point could be a user-defined object or a record.

To construct the chart, the cell C10 constructs a point consisting of the current X and Y positions of the cannonball. We then want to construct a line from all previous locations of the cannonball. This is done by specifying a time range [:-STEP()] in the cell reference. The syntax is a shorthand for [-1:-STEP()] and it selects all previous steps (STEP is a function that returns the number of the current step). The array of past values is then passed to LINE, which constructs a primitive chart shape. The GRAPH function used in cell D10 instructs Timeline to display the chart inline in the cell as a sparkline [26] without modifying the shape value. Finally, the cell D17 then overlays the individual lines and adds axes. The underlying charting library automatically aligns the chart elements and computes the range.

Extending the Spreadsheet Paradigm. The preceding example illustrated most of the extensions that Timeline makes to the core model of spreadsheets. In fact, the only significant additional feature needed by the opening Flappy Bird clone (Figure 1) is the KEYPRESS("A") function, which detects whether a key has been pressed. The game movements and collision detection are ordinary formulas and the game scene is constructed using the composable data visualization library, leveraging the IMAGE function to construct a primitive chart shape from an image URL.

The motivating example shows how the additions made in Timeline preserve the simplicity and directness of the core spreadsheet model. We add only a limited number of extensions to the formula language (the => operator and references across time). Moreover, when creating a spreadsheet in Timeline, users still start with concrete values and then extend their formulas using the drag down operation to apply a formula across a range of inputs, preserving the *from concrete cases* mode of *abstraction construction* [32].

3 Research themes

Syntactically, the only extensions that Timeline makes to the core spreadsheet model are the operator for specifying the previous (default) value => and a notation for accessing previous values A1[-1] or time ranges A1[-1:STEP()]. However, the design and implementation of Timeline draws on a number of research directions on programming languages that have previously not been connected to spreadsheets. This section provides an overview discussing the major connections. More formal treatment (§4) and case studies (§5) follow in later sections.

3.1 Synchronous languages

The notion of discrete time, as implemented in Timeline, is based on the notion of time in declarative synchronous programming languages. Synchronous languages [10] are based on the idea that time proceeds in discrete logical ticks and components react concurrently to changes in each step. The approach has been successful in areas such as signal processing and embedded control systems.

In imperative synchronous languages such as Esterel [11], programs explicitly await and emit time signals. Our approach is akin to declarative synchronous languages like Lucid and Lustre [6, 18] where expressions denote streams of values and operators and functions operate on streams in a pointwise manner. The languages additionally include a number of operators for manipulating streams. Consider the following Lustre example adapted from [10]:

```
      nat = 0 -> pre(nat) + 1;
     edge = false -> (c and not pre(c));
edgecount = if edge then pre(edgecount) + 1 else pre(edgecount)
```

The definition of `nat` illustrates two operators for manipulating streams. The `e1 -> e2` operator creates a stream that has the value of of the stream `e1` in the first time step and the value of `e2` in subsequent steps. The `pre(e)` operator constructs a stream that, in a time step t, returns the value of `e` in a time step $t-1$. The examples can be directly implemented in Timeline:

```
      /* nat */ A1 = 0 => A1[-1] + 1
     /* edge */ C1 = FALSE => AND(B1, NOT(B1[-1]))
/* edgecount */ D1 = IF(C1, D1[-1] + 1, D1[-1])
```

The Lustre examples map directly to Timeline. Although we do not explain, or formally model, Timeline in terms of streams, in practice our => and `[-1]` operations provide the same functionality as -> and `pre` in Lustre. Of course, Lustre also providers higher-order abstraction capabilities that are not present in Timeline, but those could be supported through some of the proposed abstraction mechanisms for spreadsheets [34, 40, 51].

It is also worth noting that these examples could also be easily encoded in ordinary two-dimensional spreadsheet (using rows for time steps). As illustrated by our case study from the finance domain (§ 5.4) the above way of encoding stream processing in Timeline is practically useful when working with large data sets.

3.2 Coeffect analysis

An interesting problem that Timeline shares with synchronous languages is how many past values need to be stored and whether the buffer size is bounded [16, 17]. In all the examples in § 3.1, the memory is bounded—the runtime needs to store one past value for each cell accessed via `[-1]`.

In Timeline, we adopt a static analysis based on dataflow coeffects [14, 47, 48]. A type system based on coeffects annotates expressions with context requirements, such as variable liveness or, in the case of dataflow, the number of required past values. In dataflow coeffects, type judgements of the form $\Gamma \,@\, n \vdash e : \tau$ indicate that, a given expression e can be evaluated in a variable context Γ with at least n past values for each of the variables. Structural coeffects [48] are more precise and track context requirements per-variable, by using a vector of annotations and explicit weakening, contraction and exchange rules. Figure 3 shows three of the coeffect typing rules.

The rule for variables states that accessing the current value of a variable does not require any past values. This is akin to a cell reference such as `B1`. The rule for prev e indicates that, if evaluating e requires n past values, then obtaining the previous value of e requires $n+1$ past values. Function types in the coeffect calculus are annotated with how many past values the function requires (a function $\lambda x.\text{prev}\ x$ would be annotated with 1). The rule for function application thus combines context requirements of the expression that defines the function with the context requirements

$$\frac{x : \tau \in \Gamma}{\Gamma \,@\, 0 \vdash x : \tau} \qquad \frac{\Gamma \,@\, n \vdash e : \tau}{\Gamma \,@\, n{+}1 \vdash \text{prev}\ e : \tau} \qquad \frac{\Gamma \,@\, m \vdash e_1 : \tau_1 \xrightarrow{t} \tau_2 \quad \Gamma \,@\, n \vdash e_2 : \tau_1}{\Gamma \,@\, \max(m, n+t) \vdash e_1\ e_2 : \tau_2}$$

Fig. 3. Coeffect typing rules for variables, previous value access and application.

of the evaluation of the argument, followed by the evaluation of the function (if evaluating the argument requires n past values and the function needs further t past values, then evaluating the function call requires $n + t$ past values).

In Timeline, we can only access previous values via a cell reference. Computations such as prev$(a + b)$ can be expressed by storing $a + b$ in a cell, say `A1`, and accessing `A1[-1]`. Time ranges in Timeline can be dynamic, e.g., `C10[:-STEP()]`, or even indirect, e.g., `A1[-B1]`. Such references cannot be statically checked and so the inferred number of required past values can be unbounded.

We describe the type system for checking previous value references formally in §4.4. The type system is used in two ways. First, it is used to determine the size of buffers required to store past values during spreadsheet execution. Second, it is used to check that a sufficient number of previous values is specified using the => operator (for cells with potentially unbounded number of past values, we assume that the first use of => specifies default value for any previous time step; an alternative design would be to have a separate operator for this purpose).

A basic static checking mechanism simply ensures that a cell referenced via `A1[-n]` has at least n past values, but the coeffect calculus suggests a more flexible semantics. Consider for example:

```
            /* counter */ A1 = 0 => A1[-1] + 1
      /* prev(counter) */ B1 = 0 => A1[-1]
/* prev(prev(counter)) */ C1 = B1[-1]
```

We refer to cells that require storing past values as *stateful* and cells that are always computed instantaneously as *stateless*. Note that the same distinction is made in synchronous languages, where expressions may be stateless/combinatorial/memoryless or stateful/with memory.

In the above example, `A1` and `B1` are stateful and require 1 past value each. Correspondingly, we specify one previous value for each of the cells using =>. However, the same spreadsheet could be executed by making `B1` stateless and requiring two past values of `A1`:

```
            /* counter */ A1 = 0 => 1 => A1[-1] + 1
      /* prev(counter) */ B1 = A1[-1]
/* prev(prev(counter)) */ C1 = B1[-1]
```

This alternative corresponds to the application rule of the coeffect calculus where the context requirements of a function argument and the context requirements of a function are combined. If we make `B1` stateless, we can always recompute its single previous value using the second past value of `A1`. We discuss this coeffect-based generalisation of past value access checking in §4.5.

In practice, static analysis that ensures bounded buffer sizes for stateful cells in Timeline is crucial for an efficient execution of all of the Timeline applications (§5). Unbounded buffers are only needed in situations such as the cannonball visualization (§2) where we explicitly access all past data to draw a chart, but the flappy bird clone game (Figure 1) requires only one past value of several cells, rather than storing all previously encountered states. Optionally, Timeline makes it possible to keep all past values, which enables stepping back through time for debugging.

The generalised coeffect-based execution makes it possible to trade space for time. This is theoretically interesting and it is useful in cases where, for example, a large table has a column of values that all depend on a single cell outside of the table. The optimization may reduce the memory footprint, although this would be practically useful only in limited execution environments.

3.3 Functional reactive programming

Functional reactive programming (FRP) [25, 53] is a programming paradigm for building reactive and interactive applications in functional programming languages. Given the close connection between spreadsheets and functional programming [15], it is not surprising that Timeline can draw ideas from functional reactive programming.

In FRP, programs are structured in terms of two abstractions: *behaviors* model time-varying values and *events* represent discrete event occurrences. Programs are constructed using a number of combinators that relate the two abstractions. In Fran [25] the type `Behavior a` represents streams of values of type `a`, while `Event a` represents discrete events emitting values of type `a`. The two abstractions are composed using combinators such as `untilB`:

```
-- Behave as the initial behavior, changing to the one emitted by the event
untilB :: Behavior a -> Event (Behavior a) -> Behavior a
```

The behavior `untilB start evt` has the value of `start` initially. When `evt` occurs, the value becomes that of the time-varying value emitted by the event. In Yampa [23, 31], behaviors are represented using arrows `SF a b` from input `a` to output `b`, which can be composed using a family of switch combinators such as:

```
-- Switch to a new signal function 'SF a b' whenever the specified event occurs
switch :: SF a (b, Event c) -> (c -> SF a b) -> SF a b
```

Conceptually, FRP systems are based on a continuous notion of time. Together with the rich higher-order primitives, this has historically been difficult to implement efficiently [7, 35] and some implementations adopted model with clocks [54] more akin to synchronous languages. Timeline is based on discrete time and so it eschews such implementation difficulties. However, it is still possible to view Timeline formulas as specifying time-varying behaviors. The `=>` operator is akin to `untilB` and constructs a behavior that changes when the clock tick event occurs.

Our main inspiration from functional reactive programming is for the integration of events into the programming model of Timeline. For example, in the flappy bird clone (Figure 1), the key press event controls the velocity of the bird. In the game, the cells `$O$6`, `$P$6`, and `$Q$6` contain constants that specify the gravity, jump speed and maximal velocity. The new velocity is computed as:

```
/* Velocity */ D21 = 0 => MIN($Q$6, IF(KEYPRESS("A"), -$P$6, D21[-1] + $O$6))
```

In Timeline, we can think of `IF(KEYPRESS("A"), .., ..)` as a switching operator akin to `untilB` or `switch` that changes the behavior from one time-varying value to another when the event occurs. However, because of the discrete time model, the behavior changes once and the state is stored implicitly in the history of the cell, rather than explicitly using accumulator combinators as in FRP.

3.4 Composable data visualizations

In traditional spreadsheets, visualizations are created outside of the formula language and data is selected through a user interface. This approach is not reproducible, does not enable learning [49] and is difficult to extend to data sources with the time dimension or richer visualizations.

Research that originates from grammar of graphics [56] offers a more declarative and compositional alternative [50, 55]. In systems based on the grammar of graphics, a chart is created by putting together data, scales that transform data to visual values, coordinate system, geometric marks that specify how to display data, data transformations and other aspects of visualization. For example, using the ggplot2 library [55] we can compose a chart as:

```
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_point(alpha = 0.5) + geom_smooth() + scale_y_log10()
```

The snippet separately specifies the data source diamonds, aesthetics that maps properties carat and price from the data source to X and Y coordinates, adds two geometric marks (points and smooth line) and makes the Y scale logarithmic. Grammar of graphics is very effective for scientific visualizations, but it assumes two-dimensional data source, requires a large number of primitive constructs and its range of charts is still limited.

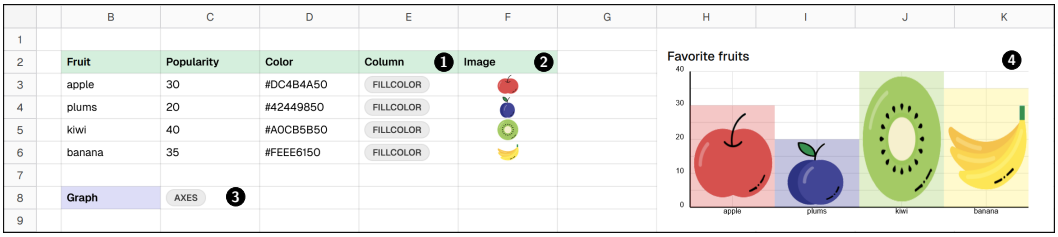

Fig. 4. Composing rich data visualization in Timeline. ❶ Columns are constructed using the fruit name and popularity. ❷ Image for each fruit is loaded, positioned and also displayed inside the table. ❸ Columns and images are composed in a single cell, ❹ which is displayed as a chart in the spreadsheet.

A more flexible approach, which fits well with the spreadsheet paradigm is based on the composition of graphical shapes. This has been developed in the context of functional programming [46], but is conceptually similar to a design prototype envisioned for spreadsheets [38].

In composable visualizations, graphical shapes are specified using domain values such as exchange rate, number of votes, country names, or population. When composed, the system infers composite scales, checks that all shapes have a compatible scale, and arranges individual shapes. This makes it possible to compose visualizations that combine different visual elements generated from data. For example, the following uses Compost.js [45] to overlay fruit images over a column chart:

```
let fruitBars =
  c.preserveAspectRatio("none", c.overlay(fruits.map(f =>
    c.overlay([
      c.fillColor(f.color,
        c.column(f.kind, f.value)),
      c.image("img/" + f.kind + ".png",
        [[f.kind, 0], 0], [[f.kind, 1], f.value]) ])
    )))
```

The snippet iterates over the data using `fruits.map`. For each data point, it creates a column and an image and overlays the two. The X scale is categorical and the image position is specified as `[f.kind,0]` and `[f.kind,1]`, which defines the left and theright end of the area allocated for the category `f.kind`. The Y scale is continuous and the values are numbers. The individual images are combined using `overlay` and the snippet uses `preserveAspectRatio` to enable scaling.

As shown in Figure 4, the same chart can be constructed in Timeline. We specify input data in a table and add two columns representing the column and image. Those are then overlaid in cell `C8`:

```
/* Construct column and image graphical shapes for each data point */
E3 = FILLCOLOR(COLUMN(B3, C3))
F3 = GRAPH(IMAGE(APPEND(B3, ".png"), POINT(COORD(B3,0),0), POINT(COORD(B3,1), 1)))

/* Overlay chart elements and specify additional chart properties */
C8 = AXES("left bottom", ASPECTRATIO("none", OVERLAY(E3:E6,F3:F6)))
```

The column is constructed using `COLUMN` which takes a categorical value (fruit name) and a numerical value (fruit popularity). It creates a rectangle that spans the whole area allocated for the category in the X dimension and ranges from 0 to the specified value in the Y dimension. When loading an image, we construct the image URL (simplified for presentation) and specify its location using `COORD`, which creates a categorical coordinate and `POINT`, which combines X and Y coordinates. Note that `COLUMN` is built on top of a primitive for filling polygon that takes a list of points.

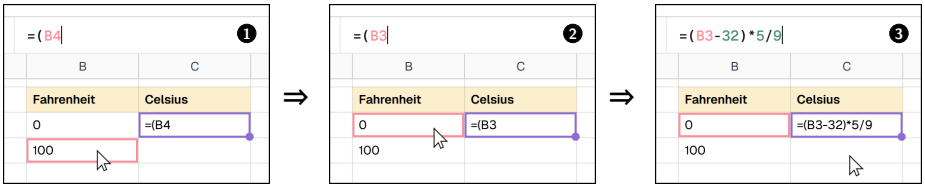

Fig. 5. Constructing a formula for Fahrenheit to Celsius conversion. ❶ After typing "(" the user clicks on a wrong cell and a reference to it is inserted, ❷ then they click on the right cell to correct the formula. ❸ After writing the rest of the formula, they click anywhere on the sheet to stop editing it.

$$
\begin{array}{lll}
(\!|\,|\!) & \rightsquigarrow (\!|\,|\!) & \text{The user starts typing and a hole is inserted} \\
 & \rightsquigarrow (\!|\mathrm{B4}|\!) & \text{The user clicks on B4 to fill the hole} \\
 & \rightsquigarrow (\!|\mathrm{B3}|\!) & \text{They correct the error with another click} \\
 & \rightsquigarrow (\mathrm{B3} - (\!|\,|\!)) & \text{As they type, the original hole is removed and a new one is added} \\
 & \rightsquigarrow (\mathrm{B3} - 32) \rightsquigarrow (\mathrm{B3} - 32) & \text{The user completes the expression} \\
 & \rightsquigarrow (\mathrm{B3} - 32) * (\!|\,|\!) \rightsquigarrow (\mathrm{B3} - 32) * 5 & \text{Hole is added after } * \text{ and then removed} \\
 & \rightsquigarrow (\mathrm{B3} - 32) * 5/(\!|\,|\!) \rightsquigarrow (\mathrm{B3} - 32) * 5/9 & \text{The same happens when adding } /5
\end{array}
$$

Fig. 6. Step-by-step editing of the Fahrenheit to Celsius formula using automatically inserted holes.

The programming model offered by composable visualizations in Timeline offers many of the user experiences prototyped in Cuscus [38]. Like Cuscus, graphical shape of a visualization can be specified using formulas in a table that supports the drag down operation, enabling non-programmers to create visualizations using tools they are already familiar with. Timeline improves on Cuscus in that it specifies shape dimensions in domain values and allows more flexible composition, such as overlaying a specified range. Visual shapes in Timeline cannot be, however, modified using direct manipulation. The encouraging results of user studies performed for Cuscus thus likely aslo apply to visualizations in Timeline.

3.5 Editing formulas with holes

An unexpected area where programming language theory becomes useful in the implementation of a spreadsheet system is formula editing. In popular spreadsheets (Excel, Google Sheets), users can click on cells while editing a formula to insert a cell reference. Interestingly, a cell reference is not inserted on every click, but only when the edit is meaningful. Moreover, click can also modify a previously inserted cell reference. A specific scenario is illustrated in Figure 5.

Precisely capturing the desired behavior is a challenge shared with recent work on collaborative editing [36]. In our case, the desired behavior follows three rules (writing • for the cursor location):

- A reference is inserted when the cursor is in a position where an expression is expected. This includes a newly created formula =•, operator parameters =10+• or a call =MIN(10,•.
- A cell reference is replaced if it was just inserted, but the user clicks on a different cell.
- In other cases, editing is completed and the clicked formula is selected. This includes, locations inside a formula such as =1•+2 or =MIN(1•,2), as well as at the end =MIN(10,A1)•.

To implement the desired behavior, we adopt the notion of holes [39] as used in the Hazel programming environment [43, 44]. Hazel introduced holes to support working with incomplete programs. A missing or unfinished sub-expression in a program is represented by a hole $(\!|\,|\!)$, which allows the rest of the program to be type checked and even run. Non-empty holes, written as $(\!|e|\!)$, represent sub-expressions of inconsistent type, i.e., a type error that needs to be resolved later.

In the Timeline formula editor, we use a (possibly non-empty) hole to represent the location where a cell reference can be inserted using a click. We adopt the Hazel notation writing ⦇⦈ for holes and we write) for tokens assumed during error recovery. The individual steps of the interaction shown in Figure 5 are listed in Figure 6 (writing ⇝ for an editing step).

The parser inserts a hole when it encounters an error at the cursor location.[1] Clicking puts a cell reference inside the hole, possibly replacing a previous reference. Holes are removed when the user enters a valid input that completes the expression. Note that a formula such as 1 + ⦇⦈ can be completed in two ways. Typing a cell reference results in 1 + A1, while clicking on the same cell results in 1 + ⦇A1⦈ and allows further clicks to change the reference.

[1]Excel supports intersection ranges written as a space-separated list of references such as A1:C3 B2:D4. These are not currently supported in Timeline, but they would be the only place where a hole needs to be inserted in the absence of a parser error. Moving the cursor to the middle of the list should insert a hole, i.e., A1:C3 ⦇⦈ B2:D4.

Column name	N	$=$	A | B | C | ...	**Value**	V	$=$	$c \mid \{V_{i,j}\}$
Row index	m	$\in$	$\mathbb{N}_1$	**Formula**	F	$=$	$V \mid R \mid f(F_1, \ldots, F_n)$
Address	a	$=$	Nm	**Sheet**	σ	$\in$	$Address \rightarrow Formula$
Range	R	$=$	$a_1 : a_2$	**Grid**	γ	$\in$	$Address \rightarrow Value$

Fig. 7. Syntax of the core Timeline calculus.

Formula evaluation: $\sigma \vdash F \Downarrow V$

$$\mathsf{size}(N_1 m_1 : N_2 m_2) = (N_2 - N_1 + 1, m_2 - m_1 + 1)$$
$$N_1 m_1 + (i, j) = (N_1 + i - 1)(m_1 + j - 1)$$

$$\frac{}{\sigma \vdash V \Downarrow V} \qquad \frac{\sigma \vdash F_i \Downarrow V_i \quad [\![f]\!](V_1, \ldots, V_n) = V}{\sigma \vdash f(F_1, \ldots, F_n) \Downarrow V} \qquad \frac{\sigma \vdash a \,!\, V}{\sigma \vdash a : a \Downarrow V}$$

$$\frac{a_1 \neq a_2 \quad \mathsf{size}(a_1 : a_2) = (n, m) \quad \forall i \in 1..n, j \in 1..m \,.\, \sigma \vdash (a_1 + (i, j)) \,!\, V_{i,j}}{\sigma \vdash a_1 : a_2 \Downarrow \{V_{i,j}\}}$$

Address dereferencing: $\sigma \vdash a \,!\, V$

$$\frac{\sigma(a) = F \quad \sigma \vdash F \Downarrow V}{\sigma \vdash a \,!\, V}$$

Sheet evaluation: $\sigma \Downarrow \gamma$

$$\frac{\forall a \in \mathsf{dom}(\sigma) \,.\, \gamma(a) = V \wedge \sigma \vdash \sigma(a) \Downarrow V}{\sigma \Downarrow \gamma}$$

Fig. 8. Reduction rules that define formula evaluation of the core Timeline calculus.

4 Calculus

In this section, we present a core calculus of Timeline. We take the *core calculus for spreadsheets* [57] as our starting point (§4.1, §4.2) and extend it with the notion of time (§4.3). We then define a system for checking references to earlier time steps (§4.4), formalize our coeffect-based optimization (§4.5) and show their soundness (§4.6).

4.1 Syntax

Figure 7 defines the syntax of the calculus. Addresses Nm consist of a column name (such as A, B, or AB) and a row index (starting from 1) and ranges are pairs of addresses. Values can be constants c or a two-dimensional arrays $\{V_{i,j}\}$ with indices $i \in 1..n$, $j \in 1..m$.

In contrast to prior work [13, 57], we omit empty values, error values and implicit error propagation. We treat errors as constants and assume that built-in functions return error when called with the error value as argument. Although we do not support spilling [57], we include arrays of values and later extend those to support ranges over time. Note also that a singleton array $\{V\}$ is distinct from a primitive value V. Formulas can be primitive values V, range references R or built-in function calls. For simplicity, cells can only be referenced through ranges so a reference `B5` are modelled as a singleton range reference `B5:B5`.

Time	t	$\in$	$\mathbb{N}_1$	**Expression**	E	$=$	$V \mid R \mid f(E_1, \ldots, E_n)$
Interval	T	$=$	$E_1 : E_2$	**Formula**	F	$=$	$E \mid E \Rightarrow F$
Range	R	$=$	$a_1 : a_2 \mid a_1 : a_2[T]$	**Timeline**	ι	$\in$	$\{1 \ldots t\} \rightarrow \mathit{Value}$
Value	V	$=$	$\ldots \mid t \mid \{V_{t,i,j}\}$	**History**	ω	$\in$	$\mathit{Address} \rightarrow \mathit{Timeline}$

Fig. 9. Syntax of the Timeline calculus with time.

Following the terminology of Williams et al. [57], we use the term *sheet* to refer to the source code and *grid* for the result of evaluating a sheet. A sheet σ is a partial function from addresses to formulas, while a grid γ is a partial function from addresses to values.

4.2 Semantics

The evaluation rules are defined in Figure 8. We follow the style of Williams et al. [57] and define evaluation with respect to a an unevaluated sheet σ. This means that dereferencing an address, defined as an auxiliary relation $\sigma \vdash a\,!\,V$ obtains the formula of the target cell and evaluates it recursively. Consequently, evaluation of cyclic references is not defined. An alternative approach [13] defines evaluation with respect to (other) evaluated values as $\gamma \vdash F \Downarrow V$ and imposes consistency requirements on the recalculation process. This rules out cyclic references for which there is no consistent value assignment, such as A1 = A1 + 1, but permits self-references such as A1 = A1.

Formula evaluation $\sigma \vdash F \Downarrow V$ is then a relation that evaluates a formula F to a value V given the sheet definition σ. A value V evaluates to itself. A function call $f(F_1, \ldots, F_n)$ evaluates its arguments and then obtains the result by applying $[\![f]\!]$, which defines the semantics of the built-in function, to the arguments. Evaluating a singleton range $a : a$ obtains a single value at the given address using dereferencing. Evaluating a range $a_1 : a_n$ iterates over all addresses in the range. The helper size returns the size of the range and $a + (i, j)$ returns an address moved by 1-based offsets i and j.

Sheet evaluation $\sigma \Downarrow \gamma$ evaluates the value at each address $a \in \mathrm{dom}(\sigma)$ defined in the sheet σ. We do not specify evaluation order, but the evaluation is still deterministic (§4.6). Our Timeline implementation builds a dependency graph between cells and evaluates them in a topological order. As discussed earlier, sheet evaluation is not defined for sheets containing a cyclic reference.

4.3 Adding the time dimension

To model Timeline, we extend the core calculus with a notion of discrete time. The extended syntax is shown in Figure 9. Time t is modelled using positive integers and, following the Timeline implementation, we refer to history using negative values $-t$. We treat time as a kind of value V. A time interval T is defined by a pair of expressions that evalue to time values. A range R can now refer to a three-dimensional range over a sheet area and a time interval. A reference to a single past value such as `F19[-1]` is modelled as a singleton range `F19:F19[-1:-1]`. Evaluating a reference to a three-dimensional range produces a three-dimensional array of values, represented as a new kind of value written as $\{V_{t,i,j}\}$.

Note that intervals can only range over historical values and so we cannot write, for example, `SUM(F19[0:-1])` to add the current and the previous value of `F19`. This simplifies the model as previous and current values are stored differently. Corresponding computations can be expressed explicitly using, for example, `F19+F19[-1]`.

To model the *followed by* operator $\Rightarrow$, we separate formulas F and expressions E. A formula can be an expression, or a formula prefixed with an initial value (i.e., a formula has zero or more prefixed values). In addition to the previously defined grid γ, we maintain the history of previously

Formula evaluation: $\sigma, \omega \vdash F \Downarrow V$

History access: $\omega \vdash a[-t] \,\text{¡}\, V$

$$\frac{\sigma, \omega \vdash F \Downarrow V}{\sigma, \omega \vdash E \Rightarrow F \Downarrow V} \qquad \frac{\sigma, \omega \vdash E \Downarrow -t \qquad \omega \vdash a[-t] \,\text{¡}\, V}{\sigma, \omega \vdash a{:}a[E{:}E] \Downarrow V} \qquad \frac{\omega(a)(t) = F}{\omega \vdash a[-t] \,\text{¡}\, V}$$

$$\frac{\begin{array}{c}\sigma, \omega \vdash E_1 \Downarrow -t_1 \qquad \sigma, \omega \vdash E_2 \Downarrow -t_2 \\ a_1 \neq a_2 \vee E_1 \neq E_2 \qquad \mathsf{size}(a_1{:}a_2) = (n, m) \qquad l = t_2 - t_1 + 1 \\ \forall i \in 1..n, j \in 1..m, t \in 1..l \,.\, \omega \vdash a_1[-t_1] + (t, i, j) \,\text{¡}\, V_{t,i,j}\end{array}}{\sigma, \omega \vdash a_1{:}a_2[-t_1{:}-t_2] \Downarrow \{V_{t,i,j}\}}$$

$$N_1 m_1[-t_1] + (t, i, j) = (N_1 + i - 1)(m_1 + j - 1)[-(t_1 + t - 1)]$$

Fig. 10. Reduction rules that define formula evaluation of the Timeline calculus with time.

evaluated values. A history ω is a partial function mapping addresses to timelines. A timeline stores a continuous range of t previous values of a cell, indexed by a time interval from 1 to t.

Formula evaluation. Figure 10 defines the extended semantics. Formula evaluation is now defined with respect to the sheet σ but also previously evaluated values represented using a history ω. The additional context can be easily added to the rules in Figure 10, but we define new rules for three-dimensional ranges and formulas prefixed with initial values. The value of $E \Rightarrow F$ is the value of F. The initial value will be used later, when initializing the history ω.

To access past values, we define an auxiliary relation $\omega \vdash a[-t] \,\text{¡}\, V$ akin to address dereferencing. However, the relation is defined with respect to the previously evaluated values and it accesses the past value from the timeline, rather than evaluating it.

In general, time ranges are specified by expressions that need to be evaluated before accessing past values. The evaluation again distinguishes the case when a range (syntactically) refers to a single value. If so, the value is obtained from the history and returned. For other ranges, we iterate over all the values and build a three-dimensional array $\{V_{t,i,j}\}$. We extend our earlier auxiliary definition of + to also support computing an offset for an address at a specified time.

Note also that some built-in functions are time-dependent. In particular, `STEP()` returns the number of the current time step. To model this, we would need to supply additional information to the semantic function $[\![-]\!]$. We omit the details for simplicity.

Sheet evaluation. Ordinary spreadsheets are evaluated to a grid of values using the $\sigma \Downarrow \gamma$ relation (Figure 8). In Timeline, the evaluation proceeds iteratively in discrete time steps for as long as the user desires (Figure 11). In each step, the sheet is evaluated using the sheet evaluation $\sigma, \omega \Downarrow \gamma$, relation which computes the current values of the formulas, with respect to a given history. This relation remains unchanged from the ordinary version, except that it now provides the current history ω to the formula evaluation.

The iterated evaluation $\sigma \Downarrow_n \gamma$ models the first n step of the evaluation of a Timeline spreadsheet. It first constructs the initial history ω_0 which is obtained by evaluating the default values (expressions before the => operator) for each cell. The number of default values corresponds to the number of `E =>` prefixes in the formula of the cell. Note that the default values are evaluated with an empty sheet and an empty history as the context. That is, an expression that defines a default value cannot itself reference other cells or their historical values (the semantics could be extended to allow referencing of other cells with greater or equal number of default values).

$\boxed{\text{Sheet evaluation: } \sigma, \omega \Downarrow \gamma}$

$$\frac{\forall a \in \mathsf{dom}(\sigma)\ .\ \gamma(a) = V \wedge \sigma, \omega \vdash \sigma(a) \Downarrow V}{\sigma, \omega \Downarrow \gamma}$$

$\boxed{\text{Iterated evaluation: } \sigma \Downarrow_n \gamma}$

$$\begin{array}{ll}
\sigma \Downarrow_n \gamma_n \iff \sigma, \omega_{n-1} \Downarrow \gamma_n & \text{where } \forall k \in 0 \ldots n-1 \\
\quad \omega_0(a) = \{1 \mapsto V_1, \ldots, n \mapsto V_n\} & \text{where } \sigma(a) = E_n \Rightarrow \ldots \Rightarrow E_1 \Rightarrow E \text{ and } \emptyset, \emptyset \vdash E_i \Downarrow V_i \\
\quad \omega_{k+1}(a)(1) = \gamma_{k+1}(a) & \text{where } \sigma, \omega_k \Downarrow \gamma_{k+1} \\
\quad \omega_{k+1}(a)(1+t) = \omega_k(t) &
\end{array}$$

Fig. 11. Sheet evaluation and iterated sheet evaluation for Timeline.

$$\frac{}{\Omega \vdash V \text{ ok}} \qquad \frac{}{\Omega \vdash a_1 : a_2 \text{ ok}} \qquad \frac{\Omega \vdash E_i \text{ ok}}{\Omega \vdash f(E_1, \ldots, E_n) \text{ ok}}$$

$$\frac{\Omega(a) \geq t}{\Omega \vdash a[-t] \text{ ok}} \qquad \frac{\forall a_i \in \mathsf{expand}(a_1 : a_2).\ \Omega \vdash a_i[-t] \text{ ok} \quad t = \max(t_1, t_2)}{\Omega \vdash a_1 : a_2[-t_1 : -t_2] \text{ ok}}$$

$$\frac{\Omega(a) = \infty}{\Omega \vdash a[E_1 : E_2] \text{ ok}} \qquad \frac{\forall a_i \in \mathsf{expand}(a_1 : a_2).\ \Omega \vdash a_i[E_1 : E_2] \text{ ok} \quad (E_1 \neq t_1) \vee (E_2 \neq t_2)}{\Omega \vdash a_1 : a_2[E_1 : E_2] \text{ ok}}$$

$$\mathsf{expand}(a_1 : a_2) = \{a_1 + (i, j), \forall i \in 1..n, j \in 1..m\} \text{ where } (n, m) = \mathsf{size}(a_1 : a_2)$$

Fig. 12. Checking

After constructing the initial history ω_0, the iterated evaluation constructs a sequence of histories $\omega_1, \ldots, \omega_{n-1}$ such that the previous value of a cell in step $k+1$ becomes the value in the evaluated grid in step k and all earlier past values are shifted by 1.

4.4 Checking timed accesses

The problem of type inference and type checking for spreadsheets has been adressed by others [3, 5] and their approaches could be directly applied to Timeline. A problem unique to Timeline is ensuring that sufficient number of past values is available.

see Figure 12

with this, we could drop (old) values from the history and also make sure there is enough default values

TODO: But we do not have infinity in ω_0 yet

4.5 Coeffect optimization

before

$$\frac{\Omega(a) \geq t_0 + t_1}{\Omega \mathbin{@} t_0 \vdash a[-t_1]\ \mathsf{ok}} \qquad \frac{\Omega(a) = \infty}{\Omega \mathbin{@} t_0 \vdash a[E_1 : E_2]\ \mathsf{ok}}$$

after

$$\frac{a \notin \mathsf{dom}(\Omega) \quad \Omega \mathbin{@} t_0 + t_1 \vdash_\sigma \sigma(a)\ \mathsf{ok}}{\Omega \mathbin{@} t_0 \vdash_\sigma a[-t_1]\ \mathsf{ok}} \qquad \frac{a \notin \mathsf{dom}(\Omega) \quad \Omega \mathbin{@} \infty \vdash_\sigma \sigma(a)\ \mathsf{ok}}{\Omega \mathbin{@} t_0 \vdash_\sigma a[E_1 : E_2]\ \mathsf{ok}}$$

??

funny history access

$$\frac{\sigma, \omega(a)(t) = F}{\omega \vdash a[-t] \mathbin{¡} V}$$

x

$$\mathsf{shift}_t(\omega)(a)(t') = \omega(a)(t + t')$$

x

$$\frac{\omega(a)(t) = \bot \qquad \sigma(a) = F \qquad \sigma, \mathsf{shift}_t(\omega) \vdash F \Downarrow V}{\sigma, \omega \vdash a[-t] \mathbin{¡} V}$$

TODO: Mention somewhere monadic/comonadic stream-based semantics

check that all items accessed at previous time step have enough past values specified

4.6 Properties and coeffect soundness

sheet evaluation is deterministic

5 Case studies

5.1 Formative interviews

5.2 Agent-bsaed

5.3 Physics

5.4 Finance

(=signal processing)

6 Related work

static analysis [21] type inference [4]

sheet defined functions [34] [51] and their semantics [13] elastic [40]

gridlets [33] and their semantics [57] [12]

References

[1] Sameera Abar, Georgios K. Theodoropoulos, Pierre Lemarinier, and Gregory M.P. O'Hare. 2017. Agent Based Modelling and Simulation tools: A review of the state-of-art software. *Computer Science Review* 24 (2017), 13–33. doi:10.1016/j.cosrev.2017.03.001

[2] Robin Abraham, Margaret Burnett, and Martin Erwig. 2009. *Spreadsheet Programming*. John Wiley & Sons, Ltd, 2804–2810. doi:10.1002/9780470050118.ecse415

[3] Robin Abraham and Martin Erwig. 2006. Type inference for spreadsheets. In *Proceedings of the 8th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming* (Venice, Italy) *(PPDP '06)*. Association for Computing Machinery, New York, NY, USA, 73–84. doi:10.1145/1140335.1140346

[4] Robin Abraham and Martin Erwig. 2006. Type inference for spreadsheets. In *Proceedings of the 8th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming* (Venice, Italy) *(PPDP '06)*. Association for Computing Machinery, New York, NY, USA, 73–84. doi:10.1145/1140335.1140346

[5] Y. Ahmad, T. Antoniu, S. Goldwater, and S. Krishnamurthi. 2003. A type system for statically detecting spreadsheet errors. In *18th IEEE International Conference on Automated Software Engineering, 2003. Proceedings*. 174–183. doi:10.1109/ASE.2003.1240305

[6] Edward A. Ashcroft and William W. Wadge. 1977. Lucid, a Nonprocedural Language with Iteration. *Commun. ACM* 20, 7 (1977), 519–526. doi:10.1145/359636.359715

[7] Patrick Bahr. 2022. Modal FRP for all: Functional reactive programming without space leaks in Haskell. *Journal of Functional Programming* 32 (2022), e15. doi:10.1017/S0956796822000132

[8] Pavel Bažant and Michaela Maršálková. 2018. A Non-Tabular Spreadsheet with Broad Applicability. In *Companion Proceedings of the 2nd International Conference on the Art, Science, and Engineering of Programming (Programming '18)*. Association for Computing Machinery, New York, NY, USA, 161–165. doi:10.1145/3191697.3214343

[9] A. Benveniste, P. Caspi, S.A. Edwards, N. Halbwachs, P. Le Guernic, and R. de Simone. 2003. The synchronous languages 12 years later. *Proc. IEEE* 91, 1 (2003), 64–83. doi:10.1109/JPROC.2002.805826

[10] A. Benveniste, P. Caspi, S.A. Edwards, N. Halbwachs, P. Le Guernic, and R. de Simone. 2003. The synchronous languages 12 years later. *Proc. IEEE* 91, 1 (2003), 64–83. doi:10.1109/JPROC.2002.805826

[11] Gérard Berry and Georges Gonthier. 1992. The Esterel Synchronous Programming Language: Design, Semantics, Implementation. *Science of Computer Programming* 19, 2 (1992), 87–152. doi:10.1016/0167-6423(92)90005-V

[12] Alexander Asp Bock, Thomas Bøgholm, Peter Sestoft, Bent Thomsen, and Lone Leth Thomsen. 2022. On the cost semantics for spreadsheets with sheet-defined functions. *J. Comput. Lang.* 69 (2022), 101103. doi:10.1016/J.COLA.2022.101103

[13] Alexander Asp Bock, Thomas Bøgholm, Peter Sestoft, Bent Thomsen, and Lone Leth Thomsen. 2020. On the semantics for spreadsheets with sheet-defined functions. *Journal of Computer Languages* 57 (2020), 100960. doi:10.1016/j.cola.2020.100960

[14] Aloïs Brunel, Marco Gaboardi, Damiano Mazza, and Steve Zdancewic. 2014. A Core Quantitative Coeffect Calculus. In *Programming Languages and Systems*, Zhong Shao (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 351–370.

[15] Margaret Burnett, John Atwood, Rebecca Walpole Djang, James Reichwein, Herkimer Gottfried, and Sherry Yang. 2001. Forms/3: A first-order visual language to explore the boundaries of the spreadsheet paradigm. *Journal of Functional Programming* 11, 2 (2001), 155–206. doi:10.1017/S0956796800003828

[16] Paul Caspi. 1992. Clocks in dataflow languages. *Theoretical Computer Science* 94, 1 (1992), 125–140. doi:10.1016/0304-3975(92)90326-B

[17] Paul Caspi, Grégoire Hamon, and Marc Pouzet. 2008. Synchronous functional programming: The Lucid Synchrone experiment. *Real-Time Systems: Description and Verification Techniques: Theory and Tools. Hermes* (2008), 28–41.

[18] Paul Caspi, Daniel Pilaud, Nicolas Halbwachs, and John Plaice. 1987. Lustre: A Declarative Language for Programming Synchronous Systems. In *Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. ACM, 178–188. doi:10.1145/41625.41641

[19] Kerry Shih-Ping Chang and Brad A. Myers. 2014. Creating Interactive Web Data Applications with Spreadsheets. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology (UIST '14)*. Association for Computing Machinery, New York, NY, USA, 87–96. doi:10.1145/2642918.2647371

[20] Kerry Shih-Ping Chang and Brad A. Myers. 2015. A Spreadsheet Model for Handling Streaming Data. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15)*. Association for Computing Machinery, New York, NY, USA, 3399–3402. doi:10.1145/2702123.2702587

[21] Tie Cheng and Xavier Rival. 2015. Static Analysis of Spreadsheet Applications for Type-Unsafe Operations Detection. In *Programming Languages and Systems*, Jan Vitek (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 26–52.

[22] Chris Clack and Lee Braine. 1997. Object-oriented functional spreadsheets. In *Proceedings of the 10th Glasgow Workshop on Functional Programming (GlaFP'97)*.

[23] Antony Courtney, Henrik Nilsson, and John Peterson. 2003. The Yampa arcade. In *Proceedings of the 2003 ACM SIGPLAN Workshop on Haskell* (Uppsala, Sweden) *(Haskell '03)*. Association for Computing Machinery, New York, NY, USA, 7–18. doi:10.1145/871895.871897

[24] Department for Energy Security & Net Zero. 2020. *The MacKay Carbon Calculator*. https://www.gov.uk/guidance/carbon-calculator Last updated 21 August 2024. Accessed: 25 February 2026.

[25] Conal Elliott and Paul Hudak. 1997. Functional reactive animation. In *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming* (Amsterdam, The Netherlands) *(ICFP '97)*. Association for Computing Machinery, New York, NY, USA, 263–273. doi:10.1145/258948.258973

[26] Leo D. Frishberg. 2011. Interactive sparklines: a dynamic display of quantitative information. In *CHI '11 Extended Abstracts on Human Factors in Computing Systems* (Vancouver, BC, Canada) *(CHI EA '11)*. Association for Computing Machinery, New York, NY, USA, 589–604. doi:10.1145/1979742.1979655

[27] Thomas A Grossman and Vijay Mehrotra. 2023. A Use Case-Engineering Resources Taxonomy for Analytical Spreadsheet Models. In *21st EuSpRIG Annual Conference "The Spreadsheet Crisis: Regaining Control"*.

[28] Sumit Gulwani, William R. Harris, and Rishabh Singh. 2012. Spreadsheet Data Manipulation Using Examples. *Commun. ACM* 55, 8 (Aug. 2012), 97–105. doi:10.1145/2240236.2240260

[29] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud. 1991. The synchronous data flow programming language Lustre. *Proc. IEEE* 79, 9 (1991), 1305–1320. doi:10.1109/5.97300

[30] Felienne Hermans, Bas Jansen, Sohon Roy, Efthimia Aivaloglou, Alaaeddin Swidan, and David Hoepelman. 2016. Spreadsheets are Code: An Overview of Software Engineering Approaches Applied to Spreadsheets. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 5. 56–65. doi:10.1109/SANER.2016.86

[31] Paul Hudak, Antony Courtney, Henrik Nilsson, and John Peterson. 2003. *Arrows, Robots, and Functional Reactive Programming*. Springer Berlin Heidelberg, Berlin, Heidelberg, 159–187. doi:10.1007/978-3-540-44833-4_6

[32] Joel Jakubovic, Jonathan Edwards, and Tomas Petricek. 2023. Technical Dimensions of Programming Systems. *The Art, Science, and Engineering of Programming* 7, 3 (2023), 13:1–13:59. doi:10.22152/programming-journal.org/2023/7/13

[33] Nima Joharizadeh, Advait Sarkar, Andrew D. Gordon, and Jack Williams. 2020. Gridlets: Reusing Spreadsheet Grids. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI EA '20)*. Association for Computing Machinery, New York, NY, USA, 1–7. doi:10.1145/3334480.3382806

[34] Simon Peyton Jones, Alan Blackwell, and Margaret Burnett. 2003. A user-centred approach to functions in Excel. In *Proceedings of the Eighth ACM SIGPLAN International Conference on Functional Programming* (Uppsala, Sweden) *(ICFP '03)*. Association for Computing Machinery, New York, NY, USA, 165–176. doi:10.1145/944705.944721

[35] Neelakantan R. Krishnaswami. 2013. Higher-order functional reactive programming without spacetime leaks. In *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming* (Boston, Massachusetts, USA) *(ICFP '13)*. Association for Computing Machinery, New York, NY, USA, 221–232. doi:10.1145/2500365.2500588

[36] Geoffrey Litt, Sarah Lim, Martin Kleppmann, and Peter van Hardenberg. 2022. Peritext: A CRDT for Collaborative Rich Text Editing. *Proc. ACM Hum.-Comput. Interact.* 6, CSCW2, Article 531 (Nov. 2022), 36 pages. doi:10.1145/3555644

[37] Steve Litt. 2017. Mitigating Spreadsheet Risk in Complex Multi-Dimensional Models in Excel. In *Proceedings of the EuSpRIG 2017 Conference "Spreadsheet Risk Management"*. European Spreadsheet Risks Interest Group (EuSpRIG), London, UK.

[38] Mariana Marasoiu, Detlef Nauck, and Alan F. Blackwell. 2019. Cuscus: An End User Programming Tool for Data Visualisation. In *End-User Development*, Alessio Malizia, Stefano Valtolina, Anders Morch, Alan Serrano, and Andrew Stratton (Eds.). Springer International Publishing, Cham, 115–131.

[39] Conor McBride. 2000. *Dependently Typed Functional Programs and their Proofs*. Ph. D. Dissertation. University of Edinburgh. LFCS Technical Report ECS-LFCS-00-419.

[40] Matt McCutchen, Judith Borghouts, Andrew D. Gordon, Simon Peyton Jones, and Advait Sarkar. 2020. Elastic sheet-defined functions: Generalising spreadsheet functions to variable-size input arrays. *Journal of Functional Programming* 30 (2020), e26. doi:10.1017/S0956796820000234

[41] Matt McCutchen, Shachar Itzhaky, and Daniel Jackson. 2016. Object Spreadsheets: A New Computational Model for End-User Development of Data-Centric Web Applications. In *Proceedings of the 2016 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2016)*. Association for Computing Machinery, New York, NY, USA, 112–127. doi:10.1145/2986012.2986018

[42] Matt McCutchen, Shachar Itzhaky, and Daniel Jackson. 2016. Object spreadsheets: a new computational model for end-user development of data-centric web applications. In *Proceedings of the 2016 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Amsterdam, Netherlands) *(Onward! 2016)*. Association for Computing Machinery, New York, NY, USA, 112–127. doi:10.1145/2986012.2986018

[43] Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. 2019. Live functional programming with typed holes. *Proc. ACM Program. Lang.* 3, POPL, Article 14 (Jan. 2019), 32 pages. doi:10.1145/3290327

[44] Cyrus Omar, Ian Voysey, Michael Hilton, Jonathan Aldrich, and Matthew A. Hammer. 2017. Hazelnut: a bidirectionally typed structure editor calculus. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages* (Paris, France) *(POPL '17)*. Association for Computing Machinery, New York, NY, USA, 86–99. doi:10.1145/3009837.3009900

[45] Tomas Petricek. 2017. Compost.js: Composable Data Visualization Library. https://github.com/compostjs/compost. Accessed 10 March 2026.

[46] Tomas Petricek. 2021. Composable data visualizations. *Journal of Functional Programming* 31 (2021), e13. doi:10.1017/S0956796821000046

[47] Tomas Petricek, Dominic Orchard, and Alan Mycroft. 2013. Coeffects: Unified Static Analysis of Context-Dependence. In *Automata, Languages, and Programming*, Fedor V. Fomin, Rūsiņš Freivalds, Marta Kwiatkowska, and David Peleg (Eds.). Springer Berlin Heidelberg, 385–397.

[48] Tomas Petricek, Dominic Orchard, and Alan Mycroft. 2014. Coeffects: a calculus of context-dependent computation. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming* (Gothenburg, Sweden) *(ICFP '14)*. Association for Computing Machinery, New York, NY, USA, 123–135. doi:10.1145/2628136.2628160

[49] Advait Sarkar and Andrew D. Gordon. 2018. How do people learn to use spreadsheets? (Work in progress). In *Proceedings of the 29th Annual Conference of the Psychology of Programming Interest Group (PPIG 2018)*. PPIG, 28–35. https://advait.org/publications-web/sarkar-2018-spreadsheet-learning

[50] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2017. Vega-Lite: A Grammar of Interactive Graphics. *IEEE Transactions on Visualization and Computer Graphics* 23, 1 (2017), 341–350. doi:10.1109/TVCG.2016.2599030

[51] Peter Sestoft and Jens Zeilund Sørensen. 2013. Sheet-Defined Functions: Implementation and Initial Evaluation. In *End-User Development*, Yvonne Dittrich, Margaret Burnett, Anders Mørch, and David Redmiles (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 88–103.

[52] The Center for Connected Learning and Computer-Based Modeling. 2025. *NetLogo Models Library*. https://ccl.northwestern.edu/netlogo/models/ Accessed: 25 February 2026.

[53] Zhanyong Wan and Paul Hudak. 2000. Functional reactive programming from first principles. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation* (Vancouver, British Columbia, Canada) *(PLDI '00)*. Association for Computing Machinery, New York, NY, USA, 242–252. doi:10.1145/349299.349331

[54] Zhanyong Wan, Walid Taha, and Paul Hudak. 2002. Event-Driven FRP. In *Practical Aspects of Declarative Languages*, Shriram Krishnamurthi and C. R. Ramakrishnan (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 155–172.

[55] Hadley Wickham. 2010. A Layered Grammar of Graphics. *Journal of Computational and Graphical Statistics* 19, 1 (2010), 3–28. doi:10.1198/jcgs.2009.07098

[56] Leland Wilkinson. 2012. *The Grammar of Graphics*. Springer Berlin Heidelberg, 375–414. doi:10.1007/978-3-642-21551-3_13

[57] Jack Williams, Nima Joharizadeh, Andrew D. Gordon, and Advait Sarkar. 2020. Higher-Order Spreadsheets with Spilled Arrays. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020 (Lecture Notes in Computer Science, Vol. 12075)*, Peter Müller (Ed.). Springer, 743–769. doi:10.1007/978-3-030-44914-8_27

[58] A.G. Yoder and D.L. Cohn. 1994. Real spreadsheets for real programmers. In *Proceedings of 1994 IEEE International Conference on Computer Languages (ICCL'94)*. 20–30. doi:10.1109/ICCL.1994.288396